

Information Processing Language (IPL) is a programming language created by Allen Newell, Cliff Shaw, and Herbert A. Simon at RAND Corporation and the Carnegie Institute of Technology about 1956. Newell had the job of language specifier-application programmer, Shaw was the system programmer, and Simon had the job of application programmer-user.

IPL included features to facilitate AI programming, specifically problem solving. such as lists, dynamic memory allocation, data types, recursion, functions as arguments, generators, and cooperative multitasking. IPL also introduced the concepts of symbol processing and list processing. Unfortunately, all of these innovations were cast in a difficult assembly-language style. Nonetheless, IPL-V (the only public version of IPL) ran on many computers through the mid 1960s.

Basics of IPL

An IPL computer has:

A set of symbols. All symbols are addresses, and name cells. Unlike symbols in later languages, symbols consist of a character followed by a number, and are written H1, A29, 9-7, 9-100.

Cell names beginning with a letter are regional, and are absolute addresses.

Cell names beginning with "9-" are local, and are meaningful within the context of a single list.

One list's 9-1 is independent of another list's 9-1.

Other symbols (e.g., pure numbers) are internal.

A set of cells. Lists are made from several cells including mutual references. Cells have several fields:

P, a 3-bit field used for an operation code when the cell is used as an instruction, and unused when the cell is data.

Q, a 3-valued field used for indirect reference when the cell is used as an instruction, and

unused when the cell is data.

SYMB, a symbol used as the value in the cell.

A set of primitive processes, which would be termed primitive functions in modern languages. The data structure of IPL is the list, but lists are more intricate structures than in many languages. A list consists of a singly linked sequence of symbols, as might be expected—plus some description lists, which are subsidiary singly linked lists interpreted as alternating attribute names and values. IPL provides primitives to access and mutate attribute value by name. The description lists are given local names (of the form 9-1). So, a list named L1 containing the symbols S4 and S5, and described by associating value V1 to attribute A1 and V2 to A2, would be stored as follows. 0 indicates the end of a list; the cell names 100, 101, etc. are automatically generated internal symbols whose values are irrelevant. These cells can be scattered throughout memory; only L1, which uses a regional name that must be globally known, needs to reside in a specific place.

IPL is an assembly language for manipulating lists. It has a few cells which are used as special-purpose registers. H1, for example, is the program counter. The SYMB field of H1 is the name of the current instruction. However, H1 is interpreted as a list; the LINK of H1 is, in modern terms, a pointer to the beginning of the call stack. For example, subroutine calls push the SYMB of H1 onto this stack.

H2 is the free-list. Procedures which need to allocate memory grab cells off of H2; procedures which are finished with memory put it on H2. On entry to a function, the list of parameters is given in H0; on exit, the results should be returned in H0. Many procedures return a Boolean result indicating success or failure, which is put in H5. Ten cells, W0-W9, are reserved for public working storage. Procedures are "morally bound" (to quote the CACM article) to save and restore the values of these cells.

There are eight instructions, based on the values of P: subroutine call, push/pop S to H0;

push/pop the symbol in S to the list attached to S; copy value to S; conditional branch. In these instructions, S is the target. S is either the value of the SYMB field if Q=0, the symbol in the cell named by SYMB if Q=1, or the symbol in the cell named by the symbol in the cell named by SYMB if Q=2. In all cases but conditional branch, the LINK field of the cell tells which instruction to execute next.

IPL has a library of some 150 basic operations. These include such operations as:

Test symbols for equality

Find, set, or erase an attribute of a list

Locate the next symbol in a list; insert a symbol in a list; erase or copy an entire list

Arithmetic operations (on symbol names)

Manipulation of symbols; e.g., test if a symbol denotes an integer, or make a symbol local

I/O operations

"Generators", which correspond to iterators and filters in functional programming. For example, a generator may accept a list of numbers and produce the list of their squares. Generators could accept suitably designed functions—strictly, the addresses of code of suitably designed functions—as arguments.

History

IPL was first utilized to demonstrate that the theorems in Principia Mathematica which were proven laboriously by hand, by Bertrand Russell and Alfred North Whitehead, could in fact be proven by computation. According to Simon's autobiography Models of My Life, this application was originally developed first by hand simulation, using his children as the computing elements, while writing on and holding up note cards as the registers which contained the state variables of the program.

IPL was used to implement several early artificial intelligence programs, also by the same

authors: the Logic Theorist (1956), the General Problem Solver (1957), and their computer chess program NSS (1958).

Several versions of IPL were created: IPL-I (never implemented), IPL-II (1957 for JOHNNIAC), IPL-III (existed briefly), IPL-IV, IPL-V (1958, for IBM 650, IBM 704, IBM 7090, Philco model 212, many others. Widely used). IPL-VI was a proposal for an IPL hardware.

A co-processor “IPL-VC” for the CDC 3600 at Argonne National Libraries was developed which could run IPL-V commands. It was used to implement another checker-playing program. This hardware implementation did not improve running times sufficiently to “compete favorably with a language more directly oriented to the structure of present-day machines”.

IPL was soon displaced by Lisp, which had much more powerful features, a simpler syntax, and the benefit of automatic garbage collection.

Legacy to computer programming

IPL arguably introduced several programming language features:

List manipulation

Property lists—but only when attached to other lists

Higher-order functions—while assembly programming had always allowed computing with the addresses of functions, IPL was an early attempt to generalize this property of assembly language in a principled way

Computation with symbols—though symbols have a restricted form in IPL (letter followed by number)

Virtual machine

Many of these features were generalized, rationalized, and incorporated into Lisp and from there into many other programming languages during the next several decades.

References

Sources

Carson, Daniel F.; Robinson, George A. (May 1966). Gyro II, A Macro-Defined System for List Processing (Report). Applied Mathematics Division, Argonne National Laboratories. ANL-7149.

Cowell, W. R.; Reed, M. C. (October 1965). A Checker-Playing Program for the IPL-VC Computer (Report). Applied Mathematics Division, Argonne National Laboratories. ANL-7109.

Hodges, Donald (May 1964). IPL-VC: A Computer System having the IPL-V Instruction Set (Report). Applied Mathematics Division, Argonne National Laboratories. ANL-6888.

Sammet, Jean E. (1969). Programming languages: history and fundamentals. Englewood Cliffs, N.J.: Prentice Hall. pp. 388–400.

Shaw, J. C.; Newell, A.; Simon, H. A.; Ellis, T. O. (1958). "A Command Structure for Complex Information Processing". Proceedings of the May 6–8, 1958, Western Joint Computer Conference: Contrasts in Computers. IRE-ACM-AIEE '58 (Western). Association for Computing Machinery. pp. 119–128. doi:10.1145/1457769.1457803. ISBN 9781450378642. {{cite conference}}: ISBN / Date incompatibility (help)

Further reading

Newell, Allen; Shaw, J. C. (1957). "Programming the Logic Theory Machine". Papers Presented at the February 26–28, 1957, Western Joint Computer Conference: Techniques for Reliability. IRE-AIEE-ACM '57 (Western). Association for Computing Machinery. pp. 230–240. doi:10.1145/1455567.1455606. ISBN 9781450378611. {{cite conference}}: ISBN / Date incompatibility (help)

Newell, Allen; Tonge, Fred M. (1960). "An Introduction to Information Processing Language V". Communications of the ACM. 3 (4). New York, NY, USA: Association for Computing Machinery: 205–211. doi:10.1145/367177.367205. ISSN 0001-0782. S2CID 16609075.

Newell, Allen; Tonge, Fred M.; et al. (1964). Information Processing Language-V Manual.

Englewood Cliffs, N.J.: Prentice Hall.

Samuel, Arthur L. (1960). "Programming Computers to Play Games". In Alt, Franz L. (ed.). *Advances in Computers* Volume 1. Vol. 1. Elsevier. pp. 165–192. doi:10.1016/S0065-2458(08)60608-7. ISBN 9780120121014. ISSN 0065-2458. {{cite book}}: ISBN / Date incompatibility (help)

External links

Allen Newell, "Biographical Memoirs", National Academy of Sciences (includes a short section on IPL)

IPL documents from BitSavers

Influence of IPL on LISP

A Common LISP interpreter for IPL-V, including a working transcription of the Logic Theory Machine